OPERATING SYSTEMS

Chapter 4: Processes

Prepared by: Yasmeen Alshraideh

ROOT TEAM
ROOT: WHERE SECURITY BEGINS

Slide 3 - Process

- Textbook uses the terms job and process almost interchangeably.
- Process – a program in execution (Execution state of a program).
- process execution must progress in sequential fashion.
- A process includes:
  - program counter
  - Stack: contains temporary data (such as method parameters, return addresses, and local variables)
  - data section: contains global variables

الكتاب بستخدم مصطلحي Job وProcess تقريبًا بنفس المعنى.

الـ Process هي برنامج قيد التنفيذ حالة تنفيذ البرنامج

تنفيذ الـ Process لازم يتقدم بشكل تسلسلي .(Sequential Fashion)

الـ Process بتتضمن:

- Program Counter: بحدد التعليمة التالية للتنفيذ.
- Stack: بحتوي بيانات مؤقتة مثل معاملات الـ Methods (Parameters) و (Return Addresses) و(Local Variables)
- Data Section: بحتوي .(Global Variables)

الـ Process مش مجرد كود؛ هي الكود مع حالته الحالية .(Program Counter + Stack + Data)

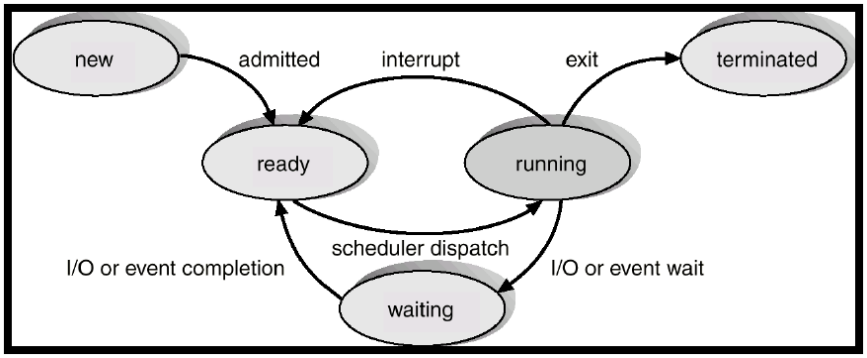
Slide 4 - Process State

- As a process executes, it changes state
  - new: The process is being created.
  - running: Instructions are being executed.
  - waiting: The process is waiting for some event to occur.
  - ready: The process is waiting to be assigned to a processer.
  - terminated: The process has finished execution.

أثناء تنفيذ الـ Process بتتغير حالتها .(State) الحالات الخمس:

- **New**: الـ Process قيد الإنشاء.
- **Running**: تعليماتها عم تنفّذ حاليًا على الـ .CPU
- **Waiting**: الـ Process عم تستنى حدوث حدث معين (Event)، مثل انتهاء عملية .I/O
- **Ready**: الـ Process جاهزة وعم تستنى يتخصص لها معالج .(CPU)
- **Terminated:** الـ Process خلصت تنفيذها.

Slide 5 - Diagram of Process State



الرسم يوضح الانتقالات (Transitions) بين الحالات:

- New → Ready: عند القبول (Admitted).
- Ready → Running: عندما يختار الـ Scheduler هالـ Process ويعطيها الـ CPU (Scheduler Dispatch).
- Running → Ready: عند حدوث مقاطعة (Interrupt)، فتنسحب الـ Process من الـ CPU وترجع لطابور الجاهزية.
- Running → Waiting: عند الانتظار لعملية I/O أو حدث (I/O or Event Wait).
- Waiting → Ready: عند اكتمال الـ I/O أو حدوث الحدث (I/O or Event Completion).
- Running → Terminated: عند الانتهاء (Exit).

الـ Process ما بتنتقل من Waiting إلى Running مباشرة؛ لازم ترجع Ready أولًا وتستنى دورها بالـ CPU.

Slide 6 - Process Control Block (PCB)

- Information associated with each process.
  - Process state (new, ready, running,…)
  - Program counter
  - CPU registers (accumulators, index registers, stack pointers,…)
  - CPU scheduling information (process priority, pointers to scheduling queues, other scheduling parameters).
  - Memory-management information
  - Accounting information
  - I/O status information

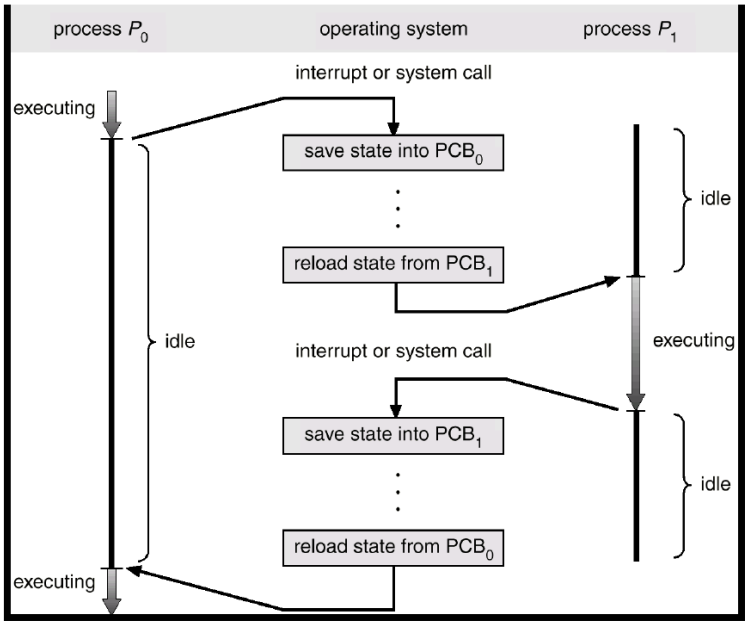
الـ PCB هو بنية بيانات بتحفظ المعلومات المرتبطة بكل Process.

محتوياته:

- Process State: حالة الـ Process(New, Ready, Running,…).
- Program Counter: عنوان التعليمة التالية.
- CPU Registers: محتوى السجلات (Accumulators وIndex Registers وStack Pointers…).
- CPU Scheduling Information: أولوية الـ Process ومؤشرات لطوابير الجدولة وبارامترات جدولة أخرى.
- Memory-Management Information: معلومات إدارة الذاكرة للـ Process.
- Accounting Information: معلومات المحاسبة (استهلاك الموارد).
- I/O Status Information: حالة الـ I/O(الأجهزة المخصصة للـ Process والملفات المفتوحة).

الـ PCB هو "هوية" الـ Process عند نظام التشغيل؛ من خلاله بيحفظ حالتها ويرجع يكمل من نفس النقطة.

Slide 8 - CPU Switch From Process to Process



الرسم يوضح انتقال الـ CPU من Process لأخرى.

- P0 بتكون Executing، ثم يحصل Interrupt or System Call فيتدخل نظام التشغيل.
- نظام التشغيل بحفظ حالة P0 في PCB0(Save State into PCB0)، وبعدها بحمّل حالة P1 من PCB1(Reload State from PCB1).
- P1 بتصير Executing، وخلال هالفترة P0 تكون Idle (ما بتنفذ).
- بعد حدوث Interrupt or System Call جديد، بينعكس الوضع: حفظ حالة P1 في PCB1(Save State into PCB1) وتحميل حالة P0 من PCB0(Reload State from PCB0)، فترجع P0 تنفّذ وP1 بتصير Idle.

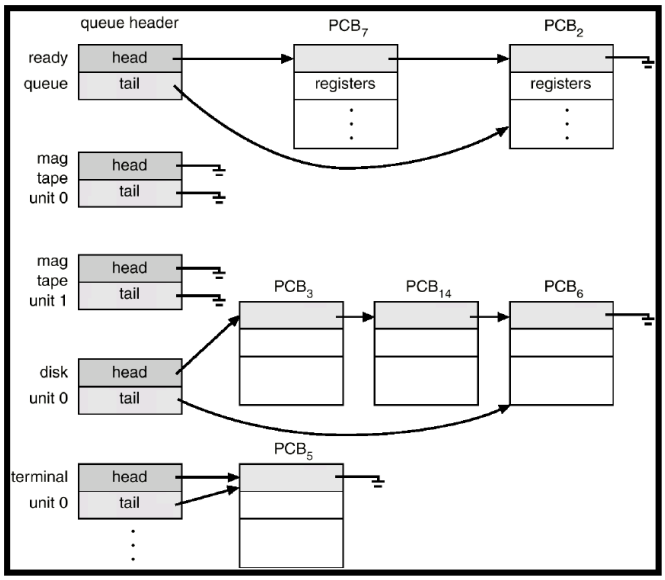
Slide 9 - Process Scheduling Queues

- Job queue – set of all computer programs that are ready to be executed in the system.
- Ready queue – set of all processes residing in main memory, ready and waiting to execute.
- Device queues – set of processes waiting for an I/O device.
- Process migration between the various queues.

(Scheduling Queues):

- Job Queue: مجموعة كل البرامج الجاهزة للتنفيذ في النظام (كما هي مذكورة في السلايد).
- Ready Queue: مجموعة كل الـ Processes الموجودة في الذاكرة الرئيسية، الجاهزة والمنتظرة التنفيذ.
- Device Queues: مجموعة الـ Processes المنتظرة لجهاز I/O معين.
- الـ Processes بتنتقل (Migration) بين هالطوابير المختلفة أثناء حياتها.

Slide 10 - Ready Queue And Various I/O Device Queues



الرسم يوضح طريقة تنظيم العمليات باستخدام Linked Lists أو مؤشرات تربط عناصر الـ Queue ببعضها.

كل Queue تحتوي على:

Head: يشير إلى أول عنصر.

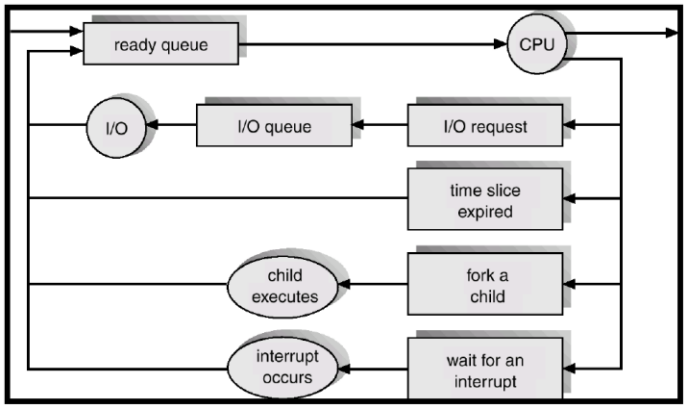
Tail: يشير إلى آخر عنصر.

PCB: يحتوي على معلومات العملية.

الأسهم بين الـ PCBs تمثل الروابط التي تسمح للنظام بالتنقل بين العمليات الموجودة في القائمة.

ليس هناك Queue واحدة لجميع العمليات؛ بل توجد Ready Queue وقوائم منفصلة للأجهزة التي تنتظرها العمليات.

Slide 11 - Representation of Process Schedulin



مسارات العمليات أثناء الجدولة:

المسار الأول CPU Execution :

تنتقل العملية من Ready Queue إلى CPU لتبدأ التنفيذ.

المسار الثاني I/O Request :

عندما تطلب العملية I/O:

1. تنتقل من CPU إلى I/O Queue.
2. تنتظر اكتمال عملية الإدخال أو الإخراج.
3. بعد انتهاء I/O ، تعود إلى Ready Queue.

المسار الثالث Time Slice Expired :

إذا انتهت الفترة الزمنية المخصصة للعملية، تعود إلى Ready Queue حتى تحصل على فرصة أخرى للتنفيذ.

المسار الرابع Fork a Child :

قد تنشئ العملية Child Process باستخدام Fork ، ثم يتم تنفيذ العملية الابنة.

المسار الخامس Interrupt :

قد تنتظر العملية حدوث Interrupt ، وعند حدوثه يمكنها متابعة مسار التنفيذ المناسب.

Slide 12 - Schedulers

- Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue.
- Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU.

يوجد نوعان أساسيان من Schedulers في هذا السلايد

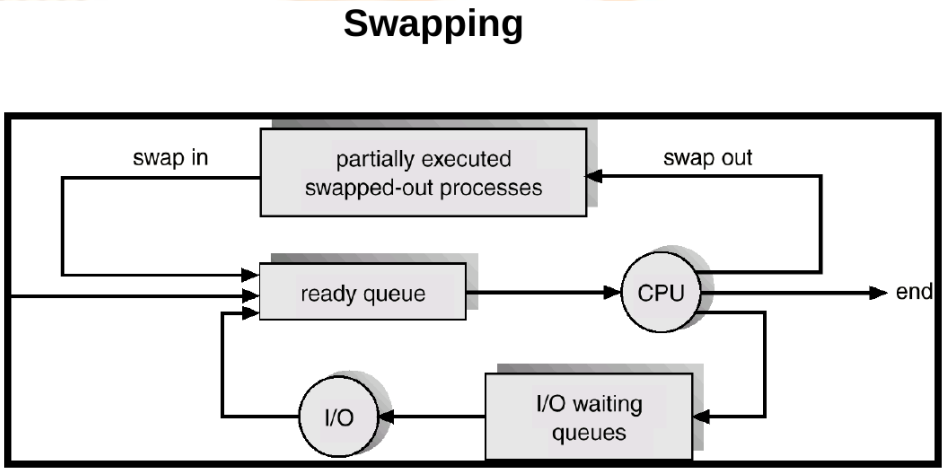
1. Long-Term Scheduler
   يحدد أي العمليات يتم إدخالها إلى Ready Queue.
   يعني يتعامل مع اختيار العمليات التي ستدخل إلى النظام للتنفيذ.
2. Short-Term Scheduler
   يختار العملية التالية التي ستنفذ على CPU .
   بمجرد وجود عدة عمليات في Ready Queue ، يقوم الـ Short-Term Scheduler باختيار واحدة منها لتشغيلها.

Slide 13 - Addition of Medium Term Scheduling

الرسم يضيف Medium-Term Scheduler إلى نموذج الجدولة، ويعرض مفهوم Swapping: عملية نقل بعض العمليات من الذاكرة الرئيسية إلى خارجها مؤقتًا، ثم إعادتها لاحقًا.

- Swap Out: نقل عمليات تم تنفيذ جزء منها إلى خارج الذاكرة الرئيسية.
- Swap In: إعادة العمليات إلى النظام.
- Ready Queue: تستقبل العمليات الجاهزة للتنفيذ.
- CPU: ينفذ العمليات.
- I/O Waiting Queues: تحتوي على العمليات المنتظرة لعمليات I/O .

دور Medium-Term Scheduling يظهر في إدارة العمليات التي يتم إخراجها مؤقتًا وإعادتها، من خلال آلية Swapping .



Slide 14 - Schedulers (Cont.)

- Short-term scheduler is invoked very frequently (milliseconds) (must be fast).
- Long-term scheduler is invoked very infrequently (seconds, minutes) (may be slow).
- The long-term scheduler controls the degree of multiprogramming.
- Processes can be described as either:
  - I/O-bound process – spends more time doing I/O than computations, many short CPU bursts.
  - CPU-bound process – spends more time doing computations; few very long CPU bursts.

الفرق في تكرار الاستدعاء:

- Short-Term Scheduler: يستدعى بشكل متكرر جدًا (كل عدة Milliseconds)، فلازم يكون سريع (Must Be Fast).

- Long-Term Scheduler: بيستدعى نادرًا (كل ثواني أو دقائق)، فممكن يكون بطيء .(May Be Slow)
- الـ Long-Term Scheduler هو اللي بتحكم بدرجة الـ Multiprogramming(كم Process بالذاكرة بنفس الوقت).

الـ Processes بتنقسم إلى نوعين:

- I/O-Bound Process: بتقضي وقتًا أكثر بعمليات الـ I/O مقارنة بالحسابات، وعندها CPU Bursts كثيرة وقصيرة.
- CPU-Bound Process: بتقضي وقتًا أكثر بالحسابات، وعندها CPU Bursts قليلة وطويلة جدًا.

Slide 15 - Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.
- Context-switch time is overhead; the system does no useful work while switching.
- Time dependent on hardware support.

عند تبديل الـ CPU إلى Process ثانية، لازم النظام يحفظ حالة الـ Process القديمة (Save State) ويحمّل الحالة المحفوظة للـ Process الجديدة .(Load State)

وقت الـ Context Switch هو Overhead(تكلفة إضافية)؛ النظام ما بعمل أي شغل مفيد أثناء التبديل.

هذا الوقت بعتمد على دعم الـ Hardware.

Slide 16 - Cooperating Processes

- Independent process cannot affect or be affected by the execution of another process.
- Cooperating process can affect or be affected by the execution of another process

- Independent Process: ما بتأثر ولا بتتأثر بتنفيذ أي Process ثانية.
- Cooperating Process: ممكن تأثر أو تتأثر بتنفيذ Process ثانية.

Slide 17 - Cooperating Processes (Advantages)

- Advantages of process cooperation
  - Information sharing (e.g. many users can share the same file, the system must provide an environment to allow concurrent access to these resources).
  - Computation speed-up (break task into subtasks and execute them in parallel in multiple processing elements)
  - Modularity (dividing the system function into separate processes or threads)
  - Convenience (e.g. a user may have many tasks on which to work at one time such as editing, compiling, printing)

فوائد تعاون الـ Processes:

- **Information Sharing**: مشاركة المعلومات، مثل مشاركة عدة مستخدمين لنفس الملف، ولازم النظام يوفر بيئة تسمح بالوصول المتزامن (Concurrent Access) لهالموارد.
- **Computation Speed-Up**: تسريع الحساب بتقسيم المهمة إلى مهام فرعية (Subtasks) وتنفيذها بالتوازي على عدة معالجات.
- **Modularity**: تقسيم وظائف النظام إلى Processes أو Threads منفصلة.
- **Convenience**: الراحة، مثل مستخدم عنده عدة مهام بنفس الوقت: تحرير وترجمة وطباعة.

Slide 18 - Producer-Consumer Problem

- Paradigm (model) for cooperating processes, producer process produces information that is consumed by a consumer process.
  - unbounded-buffer places no practical limit on the size of the buffer.
  - bounded-buffer assumes that there is a fixed buffer size.

الـ Producer-Consumer Problem هو نموذج (Paradigm) للـ Cooperating Processes: الـ Producer بنتج معلومات، والـ Consumer بستهلكها.

- Unbounded-Buffer: ما في حد عملي لحجم الـ Buffer.
- Bounded-Buffer: بفترض إن حجم الـ Buffer ثابت

Slide 19 - Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use BUFFER_SIZE-1 elements

البيانات المشتركة (Shared Data) بين الـ Producer والـ Consumer:

- BUFFER_SIZE = 10: حجم الـ Buffer.
- Item: نوع العنصر المخزّن (Struct).
- Buffer: مصفوفة (Array) بحجم BUFFER_SIZE تمثل الـ Buffer.
- In: يشير إلى أول مكان فاضي بالـ Buffer(المكان اللي رح يكتب فيه الـ Producer). قيمته الابتدائية 0.
- Out: يشير إلى أول مكان ممتلئ (المكان اللي رح يقرأ منه الـ Consumer). قيمته الابتدائية 0.

الحل صحيح، لكنه بقدر يستخدم فقط BUFFER_SIZE-1 عنصر من الـ Buffer.

يعني بـ BUFFER_SIZE = 10 بنقدر نخزن 9 عناصر فقط. (الشرح بالسلايدات التالية يوضح السبب).

Slide 20 - Bounded-Buffer – Producer Process

```
item nextProduced;
while (1) {
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
}
```

شرح كود الـ Producer:

- الحلقة الخارجية (while (1)) بتشتغل للأبد.
- الحلقة الداخلية:(((in + 1) % BUFFER_SIZE) == out)
  يعني إن الـ Buffer ممتلئ، فالـ Producer ما بعمل شي (do nothing) وبضل يستنى.
- لما يصير في مكان فاضي: بيخزن العنصر (nextProduced) في buffer [in].
- بعدها بحرك in للمكان التالي: in = (in + 1) % BUFFER_SIZE، وعملية % بتخلي المؤشر يرجع للبداية لما يوصل للنهاية (Circular Buffer).

Slide 21 - Bounded-Buffer – Consumer Process

```
item nextConsumed;
while (1) {
    while (in == out)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

شرح كود الـ Consumer:

- الحلقة الخارجية (while (1)) بتشتغل للأبد.
- الحلقة الداخلية: in == out يعني إن الـ Buffer فاضي، فالـ Consumer ما بعمل شي (do nothing) وبضل يستنى.
- لما يصير في عنصر: بيقرأه (nextConsumed = buffer[out]).
- بعدها بحرك out للمكان التالي: out = (out + 1) % BUFFER_SIZE.

Slide 22 - Interprocess Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions.
- Message system – processes communicate with each other without resorting to shared variables.
- IPC facility provides two operations:
  - send(message) – message size fixed or variable
  - receive(message)
- If P and Q wish to communicate, they need to:
  - establish a communication link between them
  - exchange messages via send/receive
- Communication link can be implemented in variety of ways (including shared memory).

الـ IPC هو آلية تسمح للـ Processes بالتواصل مع بعضها وتنسيق (Synchronize) أعمالها.

- الـ Message System: الـ Processes بتتواصل بدون استخدام متغيرات مشتركة.
- الـ IPC Facility بوفر عمليتين: send(message) وreceive(message)، وحجم الرسالة ممكن يكون ثابت أو متغير.

لو Process P وProcess Q بدهم يتواصلوا لازم:

- ينشئوا وصلة اتصال (Communication Link) بينهم.
- يتبادلوا الرسائل عن طريق send/receive.

الـ Communication Link ممكن تنفَّذ بطرق كثيرة (بما فيها Shared Memory).

يعني في الـ Message Passing ما في متغيرات مشتركة؛ كل التبادل بتم بواسطة send وreceive.

Slide 23 - Implementation Questions

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?

أسئلة تصميمية لازم تنجاوب عند تنفيذ الـ Communication Link:

- كيف تُنشأ الوصلات (Links)؟
- هل ممكن ترتبط الوصلة بأكثر من Process-ين؟
- كم وصلة ممكن تكون بين كل زوج من الـ Processes المتصلة؟
- شو سعة الوصلة (Capacity)؟
- هل حجم الرسالة اللي بتستوعبها الوصلة ثابت أم متغير؟
- هل الوصلة أحادية الاتجاه (Unidirectional) أم ثنائية الاتجاه (Bi-Directional)؟

Slide 24 - Direct Communication

- Processes must name each other explicitly:
  - send (P, message) – send a message to process P
  - receive(Q, message) – receive a message from process Q
- Properties of communication link
  - Links are established automatically.
  - A link is associated with exactly one pair of communicating processes.
  - Between each pair there exists exactly one link.
  - The link may be unidirectional, but is usually bi-directional.

في الـ Direct Communication لازم الـ Processes تسمّي بعضها بشكل صريح (Explicitly):

- send(P, message): إرسال رسالة إلى Process P.
- receive(Q, message): استقبال رسالة من Process Q.

خصائص الـ Link:

- بتتأسس تلقائيًا (Automatically).
- بترتبط بزوج واحد بالضبط من الـ Processes المتصلة.
- بين كل زوج بوجد وصلة واحدة فقط.
- ممكن تكون أحادية الاتجاه، لكنها عادةً ثنائية الاتجاه (Bi-Directional).

Slide 25 - Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports).
- Each mailbox has a unique id.
- Processes can communicate only if they share a mailbox.
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes.
  - Each pair of processes may share several communication links.
  - Link may be unidirectional or bi-directional.

في الـ Indirect Communication الرسائل بتنرسل وبتنستقبل من صناديق بريد (Mailboxes) وبتسمى كمان Ports.

- كل Mailbox إلها معرّف فريد (Unique ID).
- الـ Processes بتقدر تتواصل فقط لو بتشترك بنفس الـ Mailbox.

خصائص الـ Link:

- بتتأسس فقط لو الـ Processes بتشترك بـ Mailbox مشتركة (Common Mailbox).
- ممكن ترتبط الوصلة بعدة Processes.
- كل زوج من الـ Processes ممكن يشترك بعدة وصلات.
- الوصلة ممكن تكون أحادية أو ثنائية الاتجاه.

Slide 26 - Indirect Communication (Operations)

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:
  - send(A, message) – send a message to mailbox A
  - receive(A, message) – receive a message from mailbox A

العمليات على الـ Mailbox:

- إنشاء Mailbox جديد (Create).
- إرسال واستقبال الرسائل من خلال الـ Mailbox.
- حذف الـ Mailbox (Destroy).

الـ Primitives معرّفة كالتالي:

- send(A, message): إرسال رسالة إلى Mailbox A.
- receive(A, message): استقبال رسالة من Mailbox A.

Slide 27 - Indirect Communication (Mailbox Sharing)

- Mailbox sharing
  - P1, P2, and P3 share mailbox A.
  - P1, sends; P2 and P3 receive.
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes.
  - Allow only one process at a time to execute a receive operation.
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

مشكلة مشاركة الـ Mailbox: لو P1 وP2 وP3 بيشتركوا بـ Mailbox A، وP1 بيرسل بينما P2 وP3 بيستقبلوا، مين رح يستلم الرسالة؟

الحلول المقترحة:

- السماح بربط الوصلة مع Process-ين على الأكثر.
- السماح لـ Process واحدة فقط في نفس الوقت بتنفيذ عملية receive.
- السماح للنظام باختيار المستقبل بشكل عشوائي (Arbitrarily)، وبيتم إخطار المرسِل مين كان المستقبل.

Slide 28 - Buffering

- Queue of messages attached to the link; implemented in one of three ways.
- 1. Zero capacity – 0 messages
  - Sender must wait for receiver (rendezvous).
- 2. Bounded capacity – finite length of n messages
  - Sender must wait if link full.
- 3. Unbounded capacity – infinite length
  - Sender never waits.

الـ Buffering هو Queue من الرسائل مرتبط بال (Link)، وينفذ بواحدة من ثلاث طرق:

- Zero Capacity: سعة 0 رسالة؛ لازم المرسِل يستنى المستقبِل (Rendezvous)، يعني ما في تخزين مؤقت.
- Bounded Capacity: طول محدود (n رسالة)؛ لازم المرسِل ينتظر لو الوصلة ممتلئة.
- Unbounded Capacity: طول لانهائي؛ المرسِل ما بستنى أبدًا.

Slide 29 - Client-Server Communication

- Sockets
- Remote Procedure Calls (RPC)
- Remote Method Invocation (Java version of an RPC)

في ثلاث طرق للاتصال في أنظمة Client-Server:

- Sockets.
- Remote Procedure Calls (RPC).
- Remote Method Invocation (RMI): النسخة الخاصة بـ Java من الـ RPC.

Slide 30 - Sockets

- A socket is defined as an endpoint for communication.
- Concatenation of IP address and port
- The socket 161.25.19.8:1625 refers to port 1625 on host 161.25.19.8
- Communication consists between a pair of sockets.
- A port is an application-specific or process-specific software construct serving as a communication endpoint used by the transport layer protocols.

الـ Socket هو نقطة نهاية للاتصال (Endpoint for Communication).

بتتكون من دمج (Concatenation) عنوان IP مع رقم Port.

مثال: الـ Socket 161.25.19.8:1625 يشير إلى المنفذ 1625 على الجهاز 161.25.19.8.

الاتصال يتم بين زوج من الـ Sockets.

الـ Port هو بنية برمجية خاصة بتطبيق أو Process معينة، بتعمل كنقطة نهاية للاتصال وبتستخدمها بروتوكولات طبقة النقل (Transport Layer).

Socket = IP Address + Port.

Slide 31 - Sockets (Well-Known Ports)

- Server implementing specific services listen to well-known ports:
  - A telnet server listens to port 23.
  - An ftp server listen to port 21
  - A web (or http) server listens to port 80
- All ports below 1024 are considered well-known; we can use them to implement standard services.

الـ Servers اللي بتقدم خدمات معينة بتستمع على منافذ معروفة (Well-Known):

- Telnet Server: المنفذ 23
- FTP Server: المنفذ 21
- Web (HTTP) Server: المنفذ 80

كل المنافذ تحت 1024 تُعتبر Well-Known، وبنقدر نستخدمها لتنفيذ الخدمات القياسية (Standard Services).

telnet = 23، ftp = 21، http = 80.

Slide 32 - Sockets (Java)

- Java provides three different types of sockets:
  - Connection-oriented (TCP) sockets: implemented with the Socket class
  - Connectionless (UDP) sockets: use DatagramSockets class
  - A multicast Sockets: subclass of the DatagramSockets class

Java بتوفر ثلاثة أنواع من الـ Sockets:

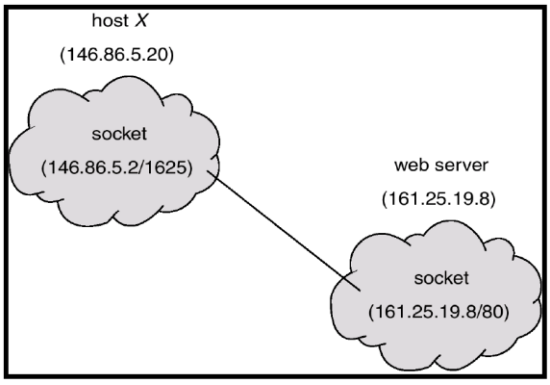
- Connection-Oriented (TCP) Sockets: بتنفذ بالـ Socket Class.
- Connectionless (UDP) Sockets: بتستخدم DatagramSockets Class.
- Multicast Sockets: هي Subclass من DatagramSockets Class.

الـ TCP بستخدم Socket Class، والـ UDP بستخدم DatagramSocket Class، أما الـ Multicast فهو Subclass من DatagramSocket.

Slide 33 - Socket Communication

الرسم يوضح اتصال بين Client وWeb Server عبر Sockets:

- Host X عنده Socket بعنوان 146.86.5.2/1625.
- الـ Web Server عنده Socket بعنوان 161.25.19.8/80، والمنفذ 80 هو منفذ الـ HTTP.
- الخط بين الـ Socketين يمثل الاتصال بينهم.



Slide 34 - Remote Procedure Calls

- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems.
- RPCs are another form of distributed communication.
- An RPC occurs when a process (or thread) calls a procedure on a remote application.
- In contrast to the IPC facility, the messages exchanged for RPC communication are well structured and are thus no longer just packets of data.

الـ Remote Procedure Call (RPC)هي آلية تسمح لعملية باستدعاء Procedure موجودة على جهاز أو نظام بعيد متصل بالشبكة.
بدل ما يكتب البرنامج تفاصيل التواصل الشبكي بالكامل، يوفر RPC مستوى من التجريد يجعل استدعاء الإجراء البعيد أقرب إلى استدعاء Procedureعادية.
تعتبر RPC أحد أشكالDistributed Communication ؛ لأنها تسمح بالتواصل بين عمليات موجودة على أنظمة مختلفة.
الرسائل المتبادلة في RPC تكون منظمة بشكل جيد(Well-Structured) ، وليست مجرد حزم بيانات عادية.

Slide 35 - RPC

- The semantics of RPCs allow a client to invoke a procedure on a remote host as it would invoke a procedure locally.

الـ Semantics تبع الـ RPC بتسمح للـ Client باستدعاء Procedure على جهاز بعيد (Remote Host) بنفس طريقة استدعاء Procedure محلية (Locally).
يعني الـ Client ما بحتاج يعرف تفاصيل الاتصال بالشبكة؛ هاي التفاصيل بتتولاها الـ Stubs

Slide 36 - RPC (Stubs)

- Stubs – client-side proxy for the actual procedure on the server.
- The client-side stub locates the port on the server and marshalls the parameters.
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server.
- Parameter marshalling involves packaging the parameters into a form which may be transmitted over a network.

الـ Stub هو وكيل (Proxy) على جهة الـ Client عن الـ Procedure الفعلية الموجودة على الـ Server.

- الـ Client-Side Stub: بحدد مكان المنفذ (Port) على الـ Server وبعمل Marshalling للمعاملات.
- الـ Server-Side Stub: بستقبل الرسالة، وبفك (Unpack) المعاملات المجمّعة، وبنفذ الـ Procedure على الـ Server.

الـ Parameter Marshalling هو تغليف (Packaging) المعاملات بصيغة قابلة للإرسال عبر الشبكة.

Slide 37 - Execution of RPC

RPC Execution Steps:

Step 1 - Client Invocation

يقوم المستخدم أو البرنامج باستدعاء Procedure على Client.

Step 2 - Client Sends Request

يقوم Client-Side Stub بتجهيز الرسالة وإرسال طلب RPC .

Step 3 - Kernel Sends Message

يتولى Kernel إرسال الرسالة عبر الشبكة.

Step 4 - Server Receives Request

يستقبل Server الرسالة ويجهزها للتنفيذ.

Step 5 - Server Executes Procedure

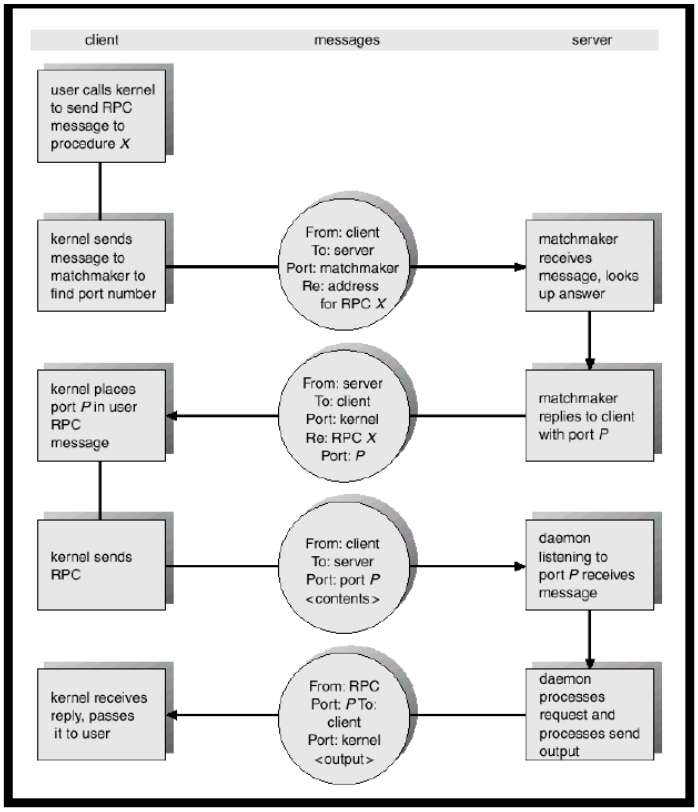
يتم تنفيذ الإجراء المطلوب على Server .

Step 6 - Server Sends Reply

يرسل Server نتيجة التنفيذ إلى Client .

Step 7 - Client Receives Reply

يستقبل Client النتيجة ويكمل التنفيذ.



Slide 38 - Remote Method Invocation

- Remote Method Invocation (RMI) is a Java mechanism similar to RPCs.
- RMI allows a Java program on one machine to invoke a method on a remote object.

الـ Remote Method Invocation (RMI) هي آلية في Java تشبه RPC.

تسمح لبرنامج Java موجود على جهاز معين باستدعاء Method موجودة داخل Remote Object.

يعني البرنامج يستطيع استدعاء Method على Object بعيد عبر الشبكة.

Slide 39 - RMI (Remote Objects)

- Objects are considered remote if they reside in a different JVM.
- The remote object may be in a different JVM on the same computer or on a remote host connected by a network.

يُعتبر الـ Object بعيدًا إذا كان موجودًا داخل JVM مختلفة عن JVM التي يعمل فيها البرنامج المستدعي.

والـ Remote Object قد يكون:

1. داخل JVM مختلفة على نفس الجهاز.
2. داخل JVM على جهاز آخر متصل بالشبكة.

Slide 40 - RPC and RMI

- RMI and RPC differ in two fundamental ways:
  - RPCs support procedural programming whereby only remote procedures or functions may be called.
  - RMI is object-based: it supports invocation of methods on remote objects.
  - RPC: data (parameters) being passed to a remote procedure are in the form of an ordinary data structure.
  - RMI: allows objects to be passed to remote methods.
- (RMI makes it possible for users to develop Java applications that are distributed across a network)

رغم التشابه بين RPC وRMI، إلا أن هناك اختلافين أساسيين.

Difference 1 - Programming Model

RPC: يعتمد علىProcedural Programming ، حيث يتم استدعاء Procedures أو Functions بعيدة.

RMI: يعتمد علىObject-Oriented Programming ، حيث يتم استدعاء Methods داخلRemote Objects .

Difference 2 - Data Passed

RPC: يتم تمرير Parameters على شكلOrdinary Data Structures .

RMI: يسمح بتمرير Objects إلىRemote Methods .

Slide 41 - RMI (Stubs and Skeletons)

- RMI implements the remote object using stubs and skeletons:
  - A stub is a proxy for the remote object, it resides with the client.
  - A stub is responsible for creating a parcel consisting of the name of the method to be invoked on the server and the marshalled parameters for the method.
  - The stub then sends this parcel to the server, where the skeleton for the remote object receives it.
  - The skeleton is responsible for unmarshalling the parameters and invoking the desired method on the server.
  - The skeleton then marshalls the return value into a parcel and returns this parcel to the client.
  - The stub unmarshalls the return value and passes it to the client.

يستخدم RMI مكونين أساسيين لتنفيذ التواصل معRemote Object :

- Stub
- Skeleton

1. Client-Side Stub : يوجد بجانبClient ، ويعمل كـ Proxy للـRemote Object .

مسؤولياته:

تجهيز اسم الـ Method المطلوب استدعاؤها.

تجهيز Parameters باستخدامMarshalling .

تجميع المعلومات داخلParcel .

إرسال Parcel إلىServer .

استقبال النتيجة.

تنفيذ Unmarshalling للنتيجة وتمريرها إلىClient .

2. **Server-Side Skeleton :** يستقبل Parcel منClient .

مسؤولياته:

استقبال الرسالة.

تنفيذ Unmarshalling للـParameters .

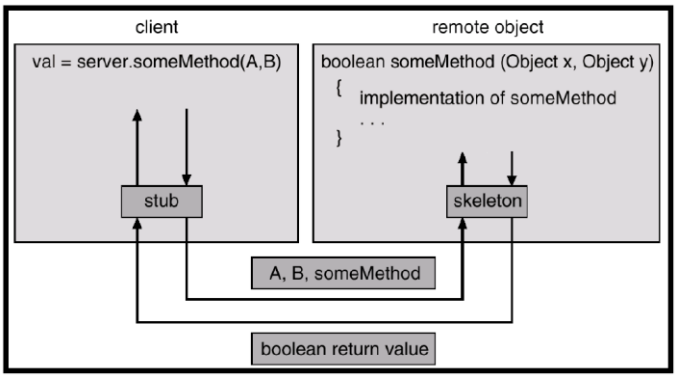
استدعاء Method المطلوبة علىServer .

تحويل Return Value باستخدامMarshalling .

إرسال النتيجة إلىClient .

Slide 42 - Marshalling Parameters

المعاملات واسم الـ Method بتنتقل من الـ Stub إلى الـ Skeleton، وقيمة الرجوع بترجع بالاتجاه المعاكس من الـ Skeleton إلى الـ Stub.



Slide 43 - RMI (Transparency)

- RMI provides a level of abstraction that makes the stubs and skeletons transparent, allowing Java developers to write programs that invoke distributed methods just as they would invoke local methods.

يوفر RMI مستوى من التجريد (Abstraction) يجعل التعامل مع Stub و Skeletonشفافًا للمبرمج.

يعني Java Developer يستطيع استدعاء Method موزعة على الشبكة بطريقة تشبه استدعاء Method محلية، دون الحاجة إلى التعامل مباشرة مع جميع التفاصيل الداخلية لعملية التواصل.

Chapter 4 – Comprehensive Review

Important Definitions

Term	Definition / Key Idea
Process	A program in execution (the execution state of a program).
Program Counter	Specifies the next instruction to execute.
Stack	Contains temporary data such as method parameters, return addresses, and local variables.
Data Section	Contains global variables.
Process State	new, running, waiting, ready, terminated.
Process Control Block (PCB)	Information associated with each process: state, program counter, CPU registers, scheduling info, memory-management info, accounting info, I/O status.
Context Switch	Saving the state of the old process and loading the saved state of the new process; pure overhead.
Job Queue	Set of all computer programs that are ready to be executed in the system.
Ready Queue	Set of all processes residing in main memory, ready and waiting to execute.
Device Queue	Set of processes waiting for an I/O device.
Long-Term Scheduler (Job Scheduler)	Selects which processes should be brought into the ready queue; controls the degree of multiprogramming.
Short-Term Scheduler (CPU Scheduler)	Selects which process should be executed next and allocates the CPU.
Swapping	Medium-term scheduling: swapping partially executed processes out of memory and back in later.
I/O-Bound Process	Spends more time doing I/O than computations; many short CPU bursts.
CPU-Bound Process	Spends more time doing computations; few very long CPU bursts.
Independent Process	Cannot affect or be affected by the execution of another process.
Cooperating Process	Can affect or be affected by the execution of another process.
Producer-Consumer Problem	A paradigm for cooperating processes: a producer produces information that a consumer consumes.
Bounded-Buffer / Unbounded-Buffer	Bounded assumes a fixed buffer size; unbounded places no practical limit on the size.
Interprocess Communication (IPC)	A mechanism for processes to communicate and to synchronize their actions.
Message System	Processes communicate with each other without resorting to shared variables, using send(message) and receive(message).
Mailbox (Port)	An object with a unique id from which messages are sent to and received (indirect communication).
Rendezvous	The sender must wait for the receiver (zero-capacity buffering).
Socket	An endpoint for communication: the concatenation of an IP address and a port.
Well-Known Ports	All ports below 1024, used for standard services (telnet 23, ftp 21, http 80).
Remote Procedure Call (RPC)	Abstracts procedure calls between processes on networked systems.
Stub	Client-side proxy for the actual procedure (or remote object) on the server.
Skeleton	In RMI, receives the parcel from the stub, unmarshalls the parameters, and invokes the method on the server.
Marshalling	Packaging the parameters into a form that can be transmitted over a network.
Remote Method Invocation (RMI)	A Java mechanism similar to RPC that lets a Java program invoke a method on a remote object.

The Five Process States

State	Meaning
New	The process is being created.
Ready	The process is waiting to be assigned to a processor.
Running	Instructions are being executed.
Waiting	The process is waiting for some event to occur.
Terminated	The process has finished execution.

Contents of the PCB

Field	Purpose
Process state	new, ready, running, waiting, terminated.
Program counter	Address of the next instruction.
CPU registers	Accumulators, index registers, stack pointers, ...
CPU scheduling information	Priority, pointers to scheduling queues, other scheduling parameters.
Memory-management information	Memory-related information of the process.
Accounting information	Resource usage information.
I/O status information	I/O state of the process.

Buffering Types

Type	Behavior
Zero capacity (0 messages)	The sender must wait for the receiver (rendezvous).
Bounded capacity (n messages)	The sender must wait if the link is full.
Unbounded capacity (infinite)	The sender never waits.

Well-Known Ports

Service	Port
Telnet server	23
FTP server	21
Web (HTTP) server	80
All ports below 1024	Considered well-known; used for standard services.

Comparison: Long-Term Scheduler vs. Short-Term Scheduler

Long-Term (Job) Scheduler	Short-Term (CPU) Scheduler
Selects which processes are brought into the ready queue.	Selects which process is executed next and allocates the CPU.
Invoked very infrequently (seconds, minutes).	Invoked very frequently (milliseconds).
May be slow.	Must be fast.
Controls the degree of multiprogramming.	Does not control the degree of multiprogramming.

Comparison: Independent vs. Cooperating Process

Independent Process	Cooperating Process
Cannot affect or be affected by other processes.	Can affect or be affected by other processes.
Needs no communication.	Needs a way to share information (IPC or shared memory).
No advantages of cooperation.	Information sharing, computation speed-up, modularity, convenience.

Comparison: Unbounded-Buffer vs. Bounded-Buffer

Unbounded-Buffer	Bounded-Buffer
No practical limit on the size of the buffer.	Assumes a fixed buffer size.
The producer never has to wait for space.	The producer waits when the buffer is full.
The consumer waits only when the buffer is empty.	The consumer waits when the buffer is empty.

Comparison: Buffer Full vs. Buffer Empty (Bounded-Buffer Code)

Buffer Full	Buffer Empty
Condition: ((in + 1) % BUFFER_SIZE) == out	Condition: in == out
The producer waits (do nothing).	The consumer waits (do nothing).
Only BUFFER_SIZE-1 elements can be used.	No element is available to consume.

Comparison: Direct vs. Indirect Communication

Direct Communication	Indirect Communication
send(P, message); receive(Q, message)	send(A, message); receive(A, message)
Processes must name each other explicitly.	Messages are directed to and received from mailboxes (ports).
Link is established automatically.	Link established only if processes share a common mailbox.
A link is associated with exactly one pair of processes.	A link may be associated with many processes.
Exactly one link between each pair.	Each pair may share several links.

Comparison: RPC vs. RMI

RPC	RMI
Supports procedural programming.	Object-based.
Only remote procedures or functions may be called.	Supports invocation of methods on remote objects.
Parameters are ordinary data structures.	Objects can be passed to remote methods.
Uses stubs (client side) and a server-side stub.	Uses stubs (client side) and skeletons (server side).
General mechanism for networked systems.	A Java mechanism similar to RPC.